

ELK330 Øving 4

John Haugstad Bø

September 2026

1 Hvordan oppgaven er løst

2 Viktige antagelser og valg

3 Resultater og diskusjon

3.1 Oppgave 1: Matematisk bevis

Når jorda roterer 360° på 24 timer vil det si at 1° tilsvarer:

$$1^\circ = \frac{24 \text{ timer}}{360^\circ} = \frac{24 \cdot 60 \text{ minutter}}{360^\circ} = 4 \underline{\underline{\text{minutter}}}$$

3.2 Oppgave 2: Bruk av formel

For å finne tidsforskjell mellom Greenwich og andre byer bruker vi formelen:

$$\Delta t = 4\Delta\lambda$$

Hvor:

Δt = Tidsforskjell

$\Delta\lambda$ = Lengdegradsforskjell

By	Lengdegrad	Lokal tid relativ til GMT
Cardiff	-3.2°	$\text{GMT} - (4 \text{ min} \cdot -3.2^\circ)$ $= \text{GMT} - 12.8 \text{ min}$
Bristol	-2.6°	$\text{GMT} - (4 \text{ min} \cdot -2.6^\circ)$ $= \text{GMT} - 10.4 \text{ min}$
Greenwich	0°	$\text{GMT} - (4 \text{ min} \cdot 0^\circ)$ $= \text{GMT}$
Bergen	5.3°	$\text{GMT} - (4 \text{ min} \cdot 5.3^\circ)$ $= \text{GMT} + 21.2 \text{ min}$
Oslo	3.8°	$\text{GMT} - (4 \text{ min} \cdot 3.8^\circ)$ $= \text{GMT} + 15.2 \text{ min}$

Table 1: Tidsforskjeller fra Greenwich for diverse byer

3.3 Oppgave 3: Forklar med egne ord

Ulike lokale tider kunne skape problemer for jernbanene fordi man ofte tok jernbanen fra en by med en tidssone til en by med en annen tidssone. For å holde oversikt på avganger og ankomster måtte man derfor drive å regne ut tidsforskjeller mellom alle de forskjellige byene basert på hvilken stasjon man befant seg på, noe som kan by på mye kaos. Det var derfor enklere at tog forholdt seg til en standardisert tid som var lik for alle de forskjellige stasjonene.

3.4 Oppgave 4: Opprette en tidsserie

Når fila leses inn er det ikke mulig å bruke `parse_dates[]` direkte på grunn av UTC-formatet, da blir tidspunktene lest inn som objekter i stedet for `datetime`-objekter. Løsningen blir å gjøre om til `datetimeindex` etter innlesing av fil via `pd.to_datetime()`.

I csv-fila er tegnet `,”` brukt som desimaltegn, mens standarden i python er å bruke `.”` som desimaltegn. For å ta høyde for dette bruker vi `decimal = ”,”` som parameter i innlesingen av csv-fila.

```

oving4 > oving4_kode.py > ...
1 # kode for øving 4
2 import pandas as pd
3
4 # Leser inn csv fil til dataframe
5 load = pd.read_csv("oving4/load_data.csv", decimal = ",")
6
7 # Gjør om til datetimeindex
8 load["Time(Local)"] = pd.to_datetime(
9     load["Time(Local)"],
10     utc = True,
11     format = "%d.%m.%Y %H:%M:%S %Z")
12
13 load = load.set_index("Time(Local)")
14
15 print(load.head())
16

```

Figure 1: Skjerm bilde av koden for oppgave 4.

```

Time(Local)
2025-12-31 23:00:00+00:00    18448.59    16662.95
2026-01-01 00:00:00+00:00    18693.66    14953.24
2026-01-01 01:00:00+00:00    18639.68    14298.07
2026-01-01 02:00:00+00:00    18502.66    13254.81
2026-01-01 03:00:00+00:00    18458.45    12678.51
PS C:\Users\johnh\Documents\GitHub\power-system-data>

```

Figure 2: Skjerm bilde av kodeutskrift etter kjøring av koden ovenfor.

En datetimeindex er en form for indeksering hvor man indekserer basert på datetime-objekter. På denne måten kan man enklere kalle på data basert på tidspunkt i stedet for å basere det på et indeks-nummer.

Tidsstempelen inneholder opprinnelig +01:00 for å vise at tidspunktet ligger en time foran UTC, men etter omgjøring til datetimeindex fjernes +01:00 og hele tidspunktet blir heller skrudd tilbake en time for å ligge på linje med UTC-tid.

3.5 Oppgave 5: Tidspunkt på første rad

Ved å kjøre koden `print(load.index[0])` får man dette resultatet:

```

johnh/Documents/GitHub/power-system-data/oving4/oving4_kode.py
2025-12-31 23:00:00+00:00

```

Figure 3: Skjerm bilde av kodeutskrift etter kjøring av koden ovenfor.

Her ser vi at tidspunktet i første rad er 23:00 den 31. desember 2025, men det er viktig å huske at dette er UTC tid. Det egentlige tidspunktet er 00:00 den 1. januar 2026, dette vet vi da det opprinnelige tidsstempelen inneholdt +01:00 for å markere en time foran UTC tid, som er tidspunktet for Norge.

3.6 Oppgave 6: Velg et tidspunkt for observasjon

15. januar 2026 er valgt som dato for lastprofil som skal plottes. 1. januar 2026 klokken 03:00 er valgt som tidspunkt for å hente ut verdier.

```
# Skrive ut verdier fra spesifikt klokkeslett
print(load.loc["2026-01-01 03:00"])

# Plotter lastprofil fra helt døgn
lastprofil = load.loc["2026-01-15"]
lastprofil.plot()
plt.grid()
plt.xlabel("Tidspunkt")
plt.ylabel("Produksjon og forbruk")
plt.savefig("oving4/lastprofil_plot")
```

Figure 4: Skjerm bilde av kode for oppgaven.

```
johnh/Documents/GitHub/power-system-data/oving4/oving4_kode.py
Production      18458.45
Consumption     12678.51
Name: 2026-01-01 03:00:00+00:00, dtype: float64
```

Figure 5: Skjerm bilde av kodeutskrift etter kjøring av koden ovenfor.

Resultatet sier at 1. januar 2026 klokken 03:00 (UTC) var produksjon på 18458.45MW og forbruk på 12678.51MW, navnet tilsvarer datetime-indexen for verdien og variabeltypen på dataene er float.

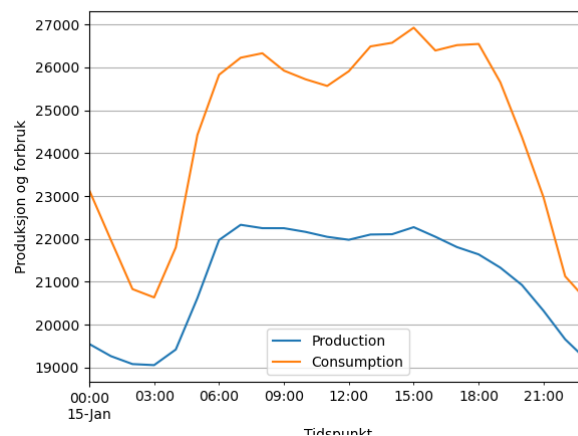


Figure 6: Plot produsert fra kjøring av koden.

3.7 Oppgave 7: Lage netto kolonne

```
31 # Lager netto kolonne
32 load["Netto"] = load["Production"] - load["Consumption"]
```

Figure 7: Kode for å lage "netto-kolonne".

3.8 Oppgave 8: Maks, min og gjennomsnittlig produksjon

```
# Finner maks, min og gjennomsnittlig produksjon
max_prod = max(load["Production"])
min_prod = min(load["Production"])
mean_prod = load["Production"].mean()

print("Maks produksjon er: ", max_prod)
print("Minimum produksjon er: ", min_prod)
print("Gjennomsnittlig produksjon er: ", round(mean_prod, 2))
```

Figure 8: Kode for å finne maks, min og gjennomsnittlig produksjon.

```
Maks produksjon er: 25320.87
Minimum produksjon er: 4078.1
Gjennomsnittlig produksjon er: 18218.51
```

Figure 9: Resultatet etter kjøring av koden ovenfor.

3.9 Oppgave 9: Maks og min nettoeffekt

```
# Finner maks og min netto produksjon
max_netto = max(load["Netto"])
min_netto = min(load["Netto"])
max_netto_tid = load["Netto"].idxmax()
min_netto_tid = load["Netto"].idxmin()

print("Maks nettoproduksjon er: ", round(max_netto, 2), " ved tidspunkt ", max_netto_tid)
print("Minimum nettoproduksjon er: ", round(min_netto, 2), " ved tidspunkt ", min_netto_tid)
```

Figure 10: Kode for å finne maks og min netto-produksjon og når disse inntreffer.

```
Maks nettoproduksjon er: 8068.22 ved tidspunkt 2026-02-22 05:00:00+00:00
Minimum nettoproduksjon er: -14059.22 ved tidspunkt 2026-04-13 15:00:00+00:00
```

Figure 11: Resultat etter kjøring av koden ovenfor.

3.10 Oppgave 10: Sum av produksjon

```
# Beregner sum
sum_prod = load["Production"].sum()

print("Summen for produksjon er: ", sum_prod)
```

Figure 12: Kode for å beregne sum av produksjon.

```
Maks nettoproduksjon er: 8068.22 ved tidspunkt 2026-02-22 05:00:00+00:00
Minimum nettoproduksjon er: -14059.22 ved tidspunkt 2026-04-13 15:00:00+00:00
```

Figure 13: Resultat etter kjøring av koden ovenfor.

Resultatet representerer summen av all produksjon for året 2026.

3.11 Oppgave 11: Produksjon og forbruk for 2026 plottet

```
57 # Plotter produksjon og forbruk for hele året som funksjon av tid
58 # plt.figure()
59 load.plot(y = ["Production", "Consumption"])
60 plt.grid(True)
61 plt.title("Produksjon og forbruk for 2026")
62 plt.xlabel("Tidspunkt")
63 plt.ylabel("Produksjon og forbruk")
64 plt.legend()
65
66 plt.savefig("oving4/prod_og_forbruk_2026.png")
```

Figure 14: Kode for å plote produksjon og forbruk for 2026.

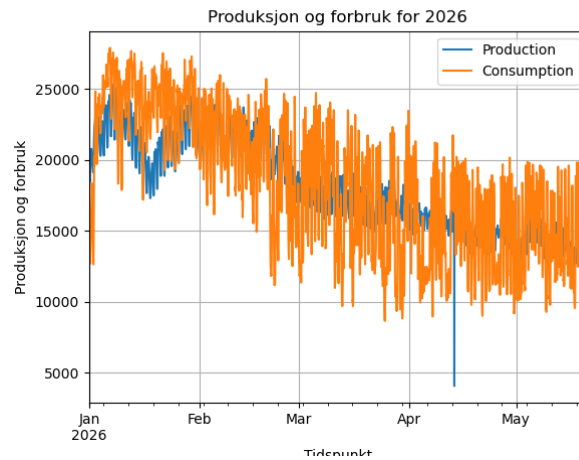


Figure 15: Plottet produsert av koden ovenfor.

3.12 Oppgave 12: Produksjon, forbruk og nettoforbruk for 2026 plottet

```
# Lager ny graf som også inkluderer netto-produksjon
load.plot()
plt.grid(True)
plt.title("Produksjon, forbruk og nettoproduksjon for 2026")
plt.xlabel("Tidspunkt")
plt.ylabel("Produksjon, forbruk og nettoproduksjon")
plt.legend()

plt.savefig("oving4/prod_forbruk_nettoprod_2026.png")
```

Figure 16: Kode for å plote produksjon, forbruk og nettoproduksjon for 2026.

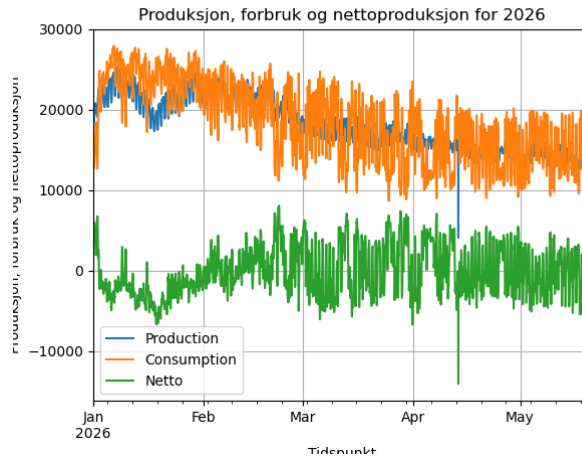


Figure 17: Plottet produsert av koden ovenfor.

3.13 Oppgave 14: Beskriv grafene

Fra grafen ser vi at forbruk er høyest i januar-februar, noe som kan forklares med lave temperaturer og høyt strømforbruk i forbindelse med oppvarming.

Vi ser at også produksjon er høyest i januar-februar. Dette gir mening da produksjonen bør holde samme nivå som forbruk i Norge, med tanke på at mesteparten av Norges produksjon også blir benyttet i Norge.

Forbruk og produksjon har i sine hovedtrekk samme mønster, med noen svingninger utenom det vanlige mønsteret. Når produksjon er høyere enn forbruk betyr det at strøm eksporteres til utlandet, mens når forbruk er høyere enn produksjon betyr det at Norge importerer strøm fra utlandet.

3.14 Oppgave 16: Git logg

```
$ git log --oneline
3a60281 (HEAD -> main) La til graf med prod, forbruk og netto. Oppdatert kode for oppgave 12
874a2c3 Oppdatert kodefil oving4_kode med kode frem til oppgave 11
09340b5 La til figur av produksjon og forbruk for 2026 plottet
edf286f La til lastprofil for 15. januar 2026 plottet
6b37bdc La til kodefil for øving 4
84129e7 La til load_data csv fil
```

Figure 18: Resultat etter kjøring av git log --oneline i Git Bash.

4 Refleksjon

Etter denne oppgaven har jeg lært at tidsserier er meget nyttig for et datasett som baserer seg på tid, spesielt da via datetimeindex systemet. Ved å benytte

pandas sammen med `datetimeindex` er det enkelt å hente ut både enkle verdier, større tidsspesifikke datasett og maks/min/gjennomsnittsverdier fra datasett.

Om git lærte jeg det samme som jeg har lært i tidligere oppgaver; Det er enkelt å holde kontroll på hvor langt man har kommet i enkelte prosjekt, man kan lett hente ut tidligere (og nåværende) versjoner av dokumenter fra hvor som helst (skole-PC og privat PC brukt i mitt tilfelle), og det hjelper med å holde en generell oversikt i større prosjekt.

Greenwich Mean Time ble nødvendig på grunn av de mange togavgangene som gikk mellom stasjoner hvor tidssonen var forskjellig. For å holde enklere kontroll på tidspunkter, hvor lang tid reiser mellom stasjoner tar, hvor langt unna tog var og lignende ble tidssonen for nærliggende byer standardisert til samme tidssone som Greenwich.